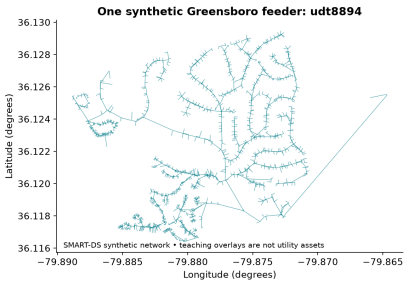
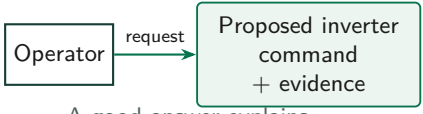
The operator's question: how much PV can this feeder accept?



One SMART-DS synthetic Greensboro feeder: `uhs20_1247-udt8894`.

An operating request

Use available PV while keeping modeled voltages and currents within their limits.



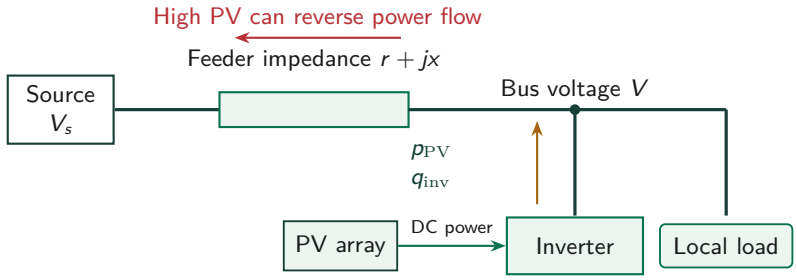
A good answer explains what was checked and what was omitted.

Predict: Which could limit additional PV: voltage, current, or inverter capability?

Explanation: Any of the three. We must inspect their margins together.

SMART-DS synthetic distribution models; Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

See the electrical path before choosing an algorithm



r: resistance; x: reactance; $\mathrm{j}^2 = -1$. $p_{\mathrm{PV}}, q_{\mathrm{inv}}$: active/reactive inverter injection.

Voltage
The line impedance turns power flow into a voltage change.

Current
Both active and reactive flow use current capacity.

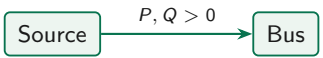
Inverter
Active and reactive commands share one kVA rating.

Gan & Low: three-phase distribution optimization (2014)

Keep net demand and inverter injection signs distinct

The feeder sees net demand
Positive P, Q mean net consumption downstream.

$$P = P_{\text{load}} - p_{\text{PV}}, \qquad Q = Q_{\text{load}} - q_{\text{inv}}.$$



P, Q: net active/reactive power; $P_{\text{load}}, Q_{\text{load}}$: demand; $p_{\text{PV}}, q_{\text{inv}}$: inverter injections. Use one consistent power base.

The inverter absorbs reactive power
$q_{\text{inv}} < 0$ increases net Q.

$$\Delta V \approx rP + xQ, \qquad V \approx V_s - \Delta V.$$



For positive line reactance x, more net reactive consumption increases the approximate drop.

Physical tradeoff
Lower voltage can require more current. We must check both.

Gan & Low: three-phase distribution optimization (2014); illustrative single-branch linearization, not a feeder measurement

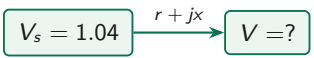
A small calculation: reactive absorption lowers high voltage

Illustrative single branch
All quantities are per unit on consistent bases.

$$V_s = 1.04, \quad r = 0.02, \quad x = 0.04,$$

$$P = -0.60.$$

Negative P means net PV export. Q is net reactive consumption. This is arithmetic for intuition, not output from the Greensboro model.



$$V \approx V_s - rP - xQ.$$

No reactive absorption: $Q = 0$
$V \approx 1.04 - 0.02(-0.60) = 1.052$ p.u.
Above the illustrative 1.05 p.u. upper bound.

Absorb reactive power: $Q = 0.20$
$V \approx 1.04 + 0.012 - 0.008 = 1.044$ p.u.
The voltage now passes that bound.

Illustrative arithmetic using a linear voltage-drop approximation; Gan & Low: three-phase distribution optimization (2014)

Voltage passes, but current can still fail

$$I \approx \frac{\sqrt{P^2 + Q^2}}{V_s}, \qquad V_{\max} = 1.05, \quad I_{\max} = 0.60 \text{ p.u.}$$

I: approximate current magnitude; P, Q: net power; $V_s = 1.04$ p.u. fixed reference. The limits are illustrative.

Action	P	Q	V	I	Checks
PV export only	−0.60	0	1.052	0.577	Voltage fails
Add absorption	−0.60	0.20	1.044	0.608	Current fails
Also reduce export	−0.55	0.20	1.043	0.563	Both pass

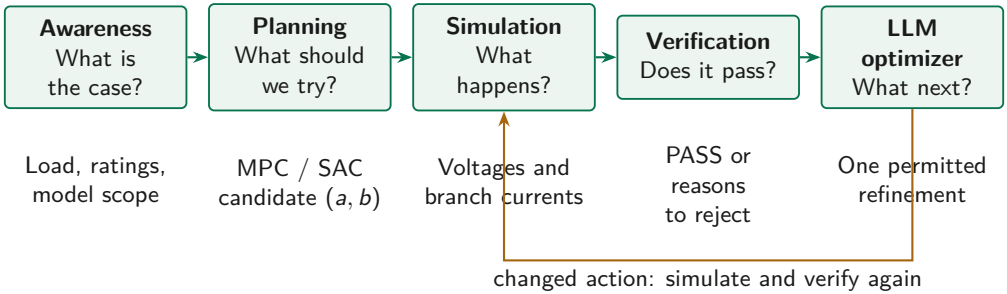
Predict: Which action would a voltage-only checker mistakenly accept?

Why we need optimization and verification

Balance active power, reactive effort and every declared limit. A single improved metric is insufficient.

Explicitly illustrative per-unit arithmetic; no measured or simulated feeder result

The five stages turn an action into reviewable evidence

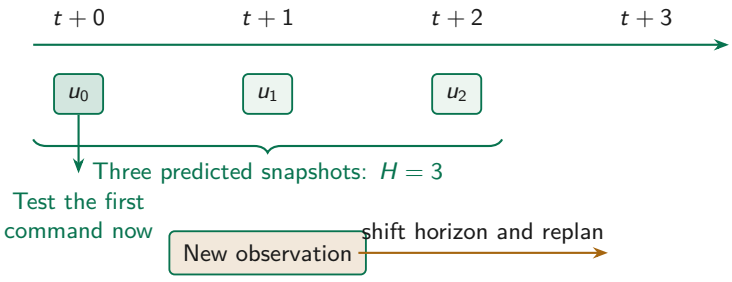


The lesson from the arithmetic

The optimizer may suggest reducing export. The numerical checker must establish whether that new command meets the declared limits.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; five-stage proposal architecture

MPC uses a forecast, then commits only the first move



$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \gamma^k \ell(x_k, u_k) \quad \text{s.t. prediction model and operating limits.}$$

u_k: control; x_k: predicted state; H: horizon; $\gamma = 0.98$: discount; ℓ: one-step cost. Here: algebraic power-flow snapshots linked by command-movement cost.

Rawlings, Mayne & Diehl: MPC (2017)

The objective makes engineering preferences explicit

One fleet command
$u = (a, b)$ is dimensionless.

$$p_i = aS_i, \qquad q_i = bS_i.$$

S_i: inverter rating (kVA); p_i: active power (kW); q_i: reactive injection (kvar).

$b < 0$ means absorption. u_{prev} is the previous command.

$$\ell = \underbrace{(1-a)^2}_{\text{unused active capability}} + \underbrace{0.05b^2}_{\text{reactive effort}} + \underbrace{10\,\text{mean}[(V-1.01)^2]}_{\text{voltage deviation}} + \underbrace{0.05\|u-u_{\text{prev}}\|_2^2}_{\text{command movement}}.$$

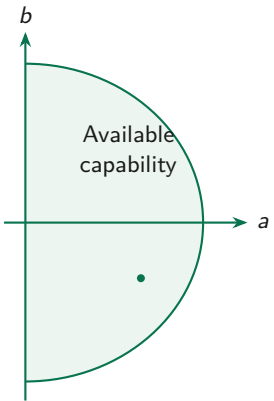
V: modeled voltage magnitudes (p.u.); ℓ: dimensionless weighted teaching cost.

Predict: If $|b|$ decreases, which cost term certainly decreases?

Explanation: Reactive effort. The voltage and movement terms can still worsen.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

The admissible region is a gate, not another preference



Network limits

$V_{\min} \leq |V_i| \leq V_{\max}, \quad |I_e| \leq I_e^{\max}.$

V_i: bus-phase voltage; I_e: branch current; bounds belong to the request.

Inverter limits

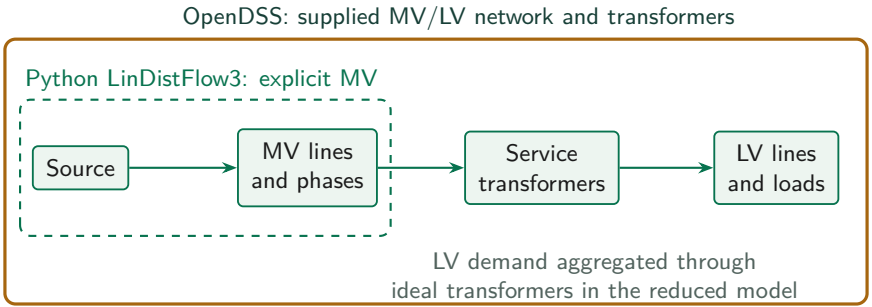
$p_i^2 + q_i^2 \leq S_i^2$ gives the capability circle; $0 \leq a \leq 1$ and declared bounds on b also apply.

Evidence limits

Finite inputs, solver completion, dispatch tracking and matching request/candidate/model identifiers.

Rawlings, Mayne & Diehl: MPC (2017); Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

The two models cover different parts of the feeder



Reduced-model PASS
Within the MV-only approximation and its checked limits.

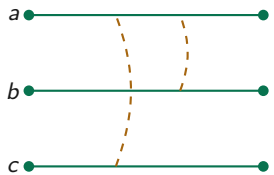
Full-model PASS
Within the broader unbalanced snapshot contract.

Comparison boundary: equations *and* constraint coverage change; this is not an accuracy-only test.

Gan & Low: three-phase distribution optimization (2014); EPRI OpenDSS documentation

Three-phase LinDistFlow keeps mutual phase coupling

Parent i Child j



Z_{ij} contains self and mutual impedance

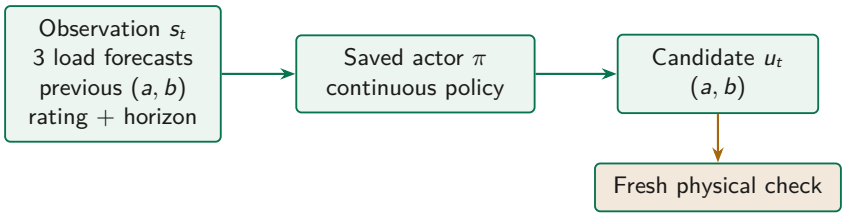
$$\mathbf{w}_j = \mathbf{w}_i - 2\,\mathrm{Re}\left[\overline{\beta} \odot \left(Z_{ij}(\beta \odot \overline{\mathbf{S}}_{ij})\right)\right].$$

$\mathbf{w}_i, \mathbf{w}_j$: squared phase voltages (V^2);
Z_{ij}: coupled phase-impedance matrix (Ω);
$\mathbf{S}_{ij}$: downstream phase powers (VA);
$\beta = (1, e^{-\mathrm{j}2\pi/3}, e^{-\mathrm{j}4\pi/3})$: nominal phase ratios.
$\overline{\cdot}$: conjugate; $\odot$: entrywise multiplication; Re: real part; $\mathrm{j}^2 = -1$.

The approximation
Nominal angle ratios and lossless branch balances. Current uses a fixed 1.03-p.u. reference; losses are diagnostic.

Gan & Low: three-phase distribution optimization (2014); exact adapter: `src/lindistflow3.py`

SAC learns a continuous action rule; checking remains separate

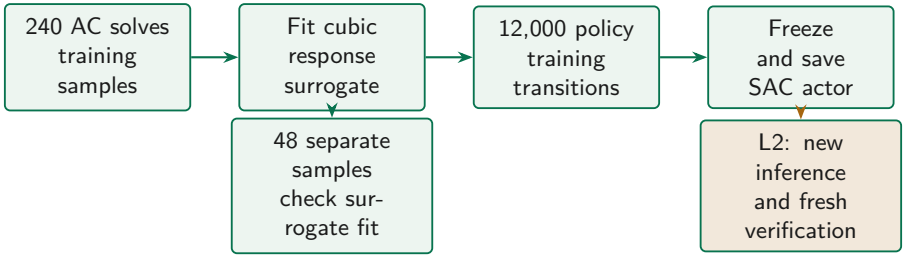


$$\max_{\pi} \mathbb{E}\left[\sum_{t\geq 0} \gamma^t \left\{ \underbrace{r(s_t, u_t)}_{\text{task reward}} + \underbrace{\alpha\, \mathcal{H}(\pi(\cdot \mid s_t))}_{\text{exploration during learning}} \right\}\right].$$

s_t: observation; u_t: action; r: reward; π: policy; $\mathcal{H}$: entropy; α: exploration weight; γ: discount; t: time step; $\mathbb{E}$: expectation over trajectories. **Executed: SAC.** PPO is a continuous-action alternative discussed here, not another trained result.

Haarnoja et al.: SAC, ICML 2018; Schulman et al.: PPO (2017)

Training provenance prevents a misleading speed claim



What the count means

12,000 transitions on the fitted surrogate, not 12,000 full AC solves.

What transfer means

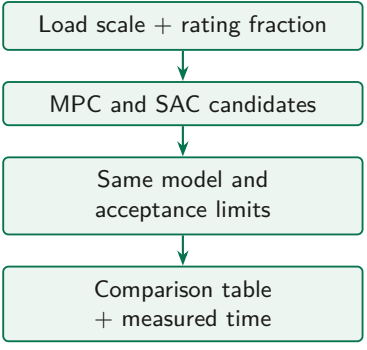
A new load or rating condition can challenge the frozen actor; L2 does not retrain it.

Haarnoja et al.: SAC, ICML 2018; executed training artifact: `Continuous_SAC_Training.ipynb`

L2 recomputes both proposals under one declared scenario

Notebook L2 Fresh MPC solve + saved SAC inference + fresh numerical checks.

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE, RATING_FRACTION)
timings['L2 fresh MPC + SAC + checks'] = time.perf_counter()
    - started
print(voltage['mode'])
display(voltage['table'])
```



Read the timer correctly

It encloses the helper call: planning, actor inference and checks. It is not isolated SAC inference latency.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

Lab: predict, change demand, then explain the response

Notebook L2 Seven minutes in pairs. Run the full cell, not only this excerpt.

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE,
     RATING_FRACTION)
```

1. Predict the effect of higher demand.
2. Run the default case.
3. Change only `LOAD_SCALE`; rerun L2.
4. Explain the limiting margin, not just the winner.

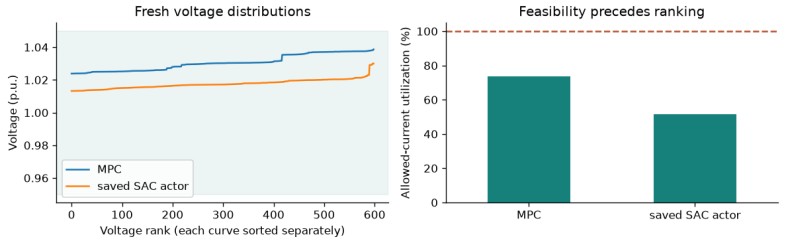
Record	Default	Changed
Feasible?	_____	_____
Objective	_____	_____
Voltage margin	_____	_____
Current margin	_____	_____

Predict: Must higher demand increase PV curtailment?

Explanation: No. Voltage and current constraints can move in different directions.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

Inspect the physical margins before ranking the controllers



Fresh L2 default	J	MV gate
MPC	0.031104	PASS
Saved SAC actor	0.436428	PASS

J: current-step teaching cost. Both actions pass this reduced-model contract. The voltage curves are sorted separately: equal rank is not equal bus identity.

Attribute the result correctly

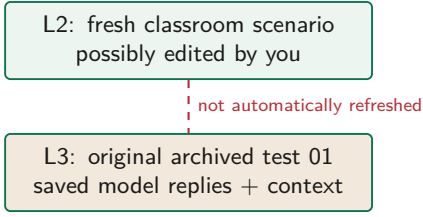
The cost gap is a controller result. It does not, by itself, measure the benefit of agent orchestration.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`

L3 opens a different, explicitly archived context

Notebook L3 Inspect original `test_01`; these are actual recorded model responses.

```
pool = lab.saved_pool('test_01')
display(pd.DataFrame(pool['responses'])[['model_id', '
    operation_id', 'reason', 'seconds', 'source']])
display(pd.DataFrame([{'operation': ', '.join(c['
    operation_ids']),
   'votes': c['vote_count'], 'share': c['vote_share'],
    'parameters': c['operation']['parameters']}
   for c in pool['pool']['candidates']]))
print('ARCHIVED ACTUAL LLM; original context:', pool['
    context']['id'])
print('Distinct physical candidates:', len(pool['pool']['
    candidates']))
print('Malformed/duplicate votes rejected:', len(pool['
    pool']['rejected']))
```



Read the provenance

`reason`: exact public response.
`seconds`: recorded call time.
Printing the table makes no new LLM call.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

Read the actual replies, then challenge their claims

Exact public excerpts, archived original test 01. Llama is the model family; Ollama is the local runtime.

Llama 3.1 8B 1.867 s
Trial: increase a by 0.0025.

Increasing active participation by 0.0025 may help reduce curtailment and improve the overall performance.

Llama 3.2 3B 0.905 s
Trial: reduce $|b|$ by 10%.

Reducing reactive effort magnitude by ten percent may help improve the overall system performance and reduce curtailment

Qwen 2.5 7B 1.844 s
Trial: reduce $|b|$ by 10%.

may help improve voltage stability without significantly degrading active power performance

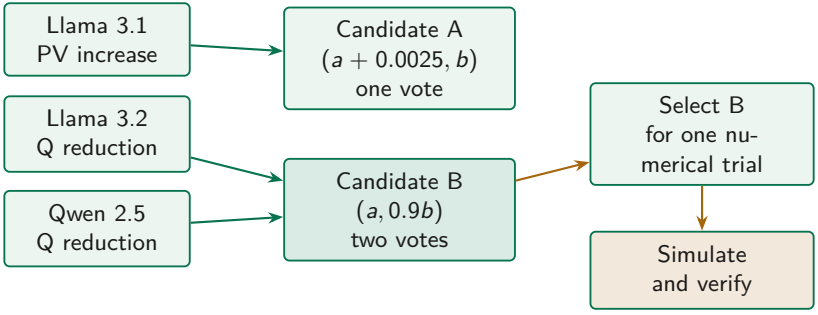
Predict: Which phrases need evidence beyond a plausible explanation?

Reactive effort $\neq$ curtailment
Reducing $|b|$ does not itself prove higher active generation.

Snapshot $\neq$ stability proof
Steady-state power flow does not establish voltage stability.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

Canonicalize actions before counting votes



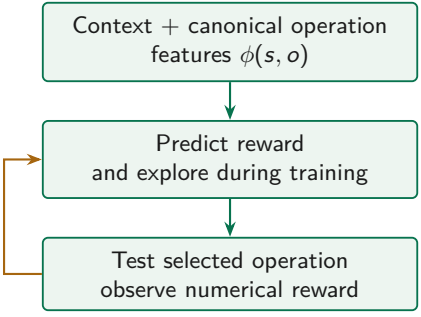
a, b: incumbent fleet command. Tool name and parameters identify the same physical operation; prose style does not create a new action.

A majority is a selection rule

Correlated language models can share an error. Agreement is not a calibrated probability of safety.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

LinUCB learns which proposed operation is worth testing



$$q(o \mid s) = \underbrace{\theta^\top \phi(s,o)}_{\text{predicted reward}} + \underbrace{\beta\sqrt{\phi(s,o)^\top A^{-1}\phi(s,o)}}_{\text{training exploration bonus}},$$

$$o^* = \arg\max_o q(o \mid s).$$

s: evidence context; o: canonical operation; ϕ: features; θ: learned coefficients; A: regularized feature information; β: exploration weight; q: score; o^*: selected operation.

Frozen classroom evaluation

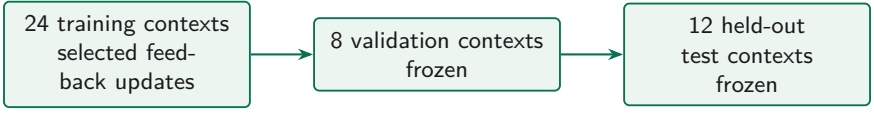
Use predicted reward only, with no updates. All LLM replies already exist: this learner does not choose which model to call or train LLM weights.

Li et al.: LinUCB, WWW 2010; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

Reward learning and physical acceptance have different jobs

$$r = \begin{cases} \min\left(1, \dfrac{\max(0, J_{\text{inc}} - J_{\text{trial}})}{0.01}\right) - 0.01, & \text{trial passes,} \\ -1.01, & \text{trial rejected.} \end{cases}$$

r: post-trial reward; $J_{\text{inc}}, J_{\text{trial}}$: incumbent and trial current-step costs. The 0.01 charge is a fixed trial penalty, *not seconds*.



Predict: The learner predicts improvement, but the trial violates current. Accept?

Explanation: Reject the trial. Keep a valid incumbent or escalate; reward does not grant permission.

Li et al.: LinUCB; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

L4: verify the frozen choice with a new numerical trial

Notebook L4 Three minutes: run the full cell and explain the output.

```
decision = pool['selection']
print('Frozen:', pool['frozen'], '| selected-feedback training
    updates:', pool['training_updates'])
print('Bandit operation:', decision.operation_id, '| predicted
    reward:', round(decision.predicted_reward, 4))
print('Same canonical choice as majority:', decision.
    candidate_id == pool['vote_id'])
started = time.perf_counter()
selected_trials = lab.trial_saved_selection(pool)
timings['L4 new verification of archived choices'] = time.
    perf_counter() - started
display(selected_trials['table'])
```

Before the trial
Which canonical operation did the learner choose? Does it match the majority?

After the trial
Did the fresh gate pass? What current-step cost was obtained?

Default: learner and vote select Q reduction; original-context $J \approx 0.031082$.

This evaluates archived original choices in their original context. Do not subtract this cost from a changed L2 scenario's cost.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

A fair experiment can find no learned advantage

Recorded held-out MV pilot: 12 scenarios; lower J is better under the same reduced-model contract.

Selection method	Mean J	Same-action full AC PASS
Fixed numerical refinement	0.03400656	0/12
Majority vote	0.03398479	0/12
Frozen learned selector	0.03398479	0/12

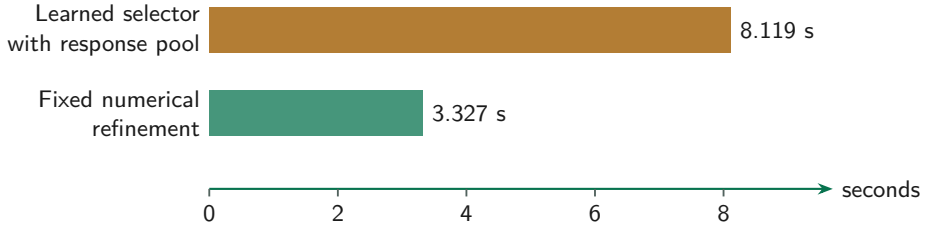
Observed
Vote and learner tie on every held-out case.

Diagnostic
A posthoc fixed Q-reduction rule reproduces their choices without LLM calls.

What this supports
Reproducible trials and explicit checking are demonstrated. Superior control from extra LLMs or a learned selector is not established.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

Report the latency of the entire decision path



Recorded component times, reconstructed serial accounting for the held-out MV pilot. These are not guarantees for a deployed control room.

Count all work
Every pool selector pays for all three LLM responses, plus numerical trials and checks.

Separate timing boundaries
Training, model loading, token generation, actor inference and end-to-end execution are different quantities.

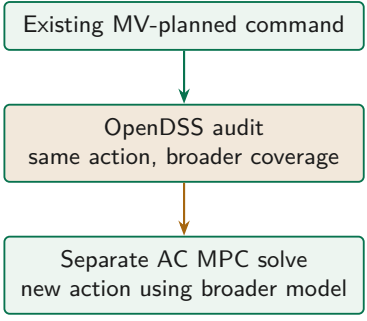
Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`; Ollama chat API timing fields

L5 separates same-action auditing from AC replanning

Notebook L5 Default: fresh MV audit. Optional: full AC audit and a separate AC MPC solve.

```
started = time.perf_counter()
scope_audit = lab.backend_audit(pool, run_opendss=
    RUN_OPENDSS)
timings['L5 selected model scope checks'] = time.
    perf_counter() - started
display(scope_audit['table'])
print(scope_audit['optional_mode'])
print('An MV-only PASS cannot acknowledge a request
    requiring full MV/LV coverage.')
```

Enable `RUN_OPENDSS` in L0 for the optional path. Run OpenDSS evaluations sequentially.

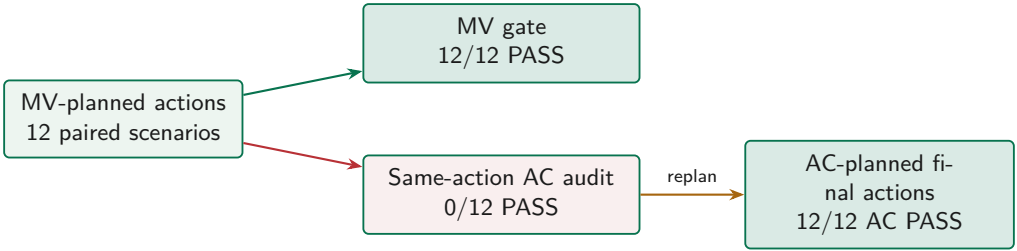


Two questions

Does the old recommendation survive the audit? What would a planner using the broader model recommend instead?

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`; EPRI OpenDSS documentation

A reduced-model PASS can fail the full-feeder audit



What changed
Numerical equations and network/constraint coverage. The selector remained frozen.

What did not follow
This does not prove an LLM caused the better physical result, nor does PASS constitute human approval.

Separately recorded 12-case pilot; not the output of one classroom L5 execution.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

The AC comparison still needs a strong numerical baseline

Recorded paired AC pilot, 12 scenarios.

Method	Mean current-step J
MPC first action	0.47172417
Fixed refinement	0.47112138
Uniform pool selection	0.47110513
Majority vote	0.47138488
Frozen learned selection	0.47138488

Interpretation

Vote and learner tie. The learned selector does not beat the fixed or uniform comparators here.

Check the raw policy too

Only 1/12 raw SAC actions passes the AC gate in this pilot. A fast proposal is not automatically usable.

Why this is a scientific result

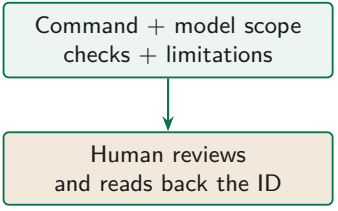
Hold scenario, feasible set, budget and evaluation boundary constant. Attribute changes to the controller, model, information or orchestration mechanism that actually changed.

Do not compare AC and MV objective means as a pure solver ranking: the modeled voltage sets differ.

Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`

The operator receives a scoped recommendation and its evidence

Stage	Returned evidence in this lesson
Awareness	Load, rating fraction, requested scope, revision
Planning	MPC and SAC candidate commands
Simulation	New voltages, currents and objective
Verification	PASS / failures; candidate-bound model evidence
LLM optimizer	Permitted trial and recorded public rationale



Full scope requested?
An MV-only PASS remains HOLD for a request requiring full MV/LV coverage.

Demonstrated benefit and boundary
The workflow makes assumptions, trials and failures inspectable. Extra LLMs did not demonstrate better control here. Readback is advisory, not equipment dispatch.

Executed `Classroom_Labs.ipynb`; `src/classroom_labs.py`, L1–L5 and L12; Recorded `LLM_Response_Selection.ipynb`; raw calls in `results/ensemble_voltage_pilot/`